

LAPORAN PRAKTIKUM

STRUKTUR DATA

“SORTING”



disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena berkat rahmat dan karunia-Nya, laporan praktikum dengan judul “SORTING” dapat diselesaikan tepat waktu.

Laporan ini disusun untuk memenuhi salah satu tugas mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran dalam memahami konsep dasar pemrograman khususnya materi Sorting pada Bahasa Java.

Pada kesempatan ini, penulis menyampaikan terimakasih kepada:

1. Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Praktikum Struktur Data yang telah memberikan bimbingan dan arahan.
2. Kak Zahran Azaria Anvaya selaku asisten praktikum kelas A yang telah membantu pelaksanaan praktikum.
3. Teman-teman mahasiswa yang telah membantu dan memberi dukungan serta berdiskusi bersama dalam menyelesaikan laporan ini.

Penulis menyadari bahwa laporan ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan menambah wawasan mengenai pemrograman serta struktur data pada Java.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	2
BAB II	3
2.1 Langkah Kerja Praktikum	3
2.2 Analisis Hasil dan Pembahasan	16
BAB III.....	18
3.1 Kesimpulan	18
DAFTAR PUSTAKA	
KATA PENGANTAR.....	i

BAB I PENDAHULUAN

1.1 Latar Belakang

Pada praktikum pekan ke-7 ini, mahasiswa mempelajari salah satu materi dasar dalam struktur data, yaitu **algoritma sorting**. Sorting digunakan untuk mengurutkan data agar lebih terstruktur dan mudah dipahami. Dalam praktikum ini, proses sorting diterapkan pada data bertipe **integer** menggunakan tiga algoritma, yaitu **Bubble Sort, Insertion Sort, dan Selection Sort**, yang ditulis menggunakan bahasa pemrograman Java.

Pada codingan pertama hingga ketiga, proses sorting dilakukan melalui program berbasis **console**, sehingga mahasiswa dapat melihat perbedaan kondisi data sebelum dan sesudah dilakukan pengurutan. Sedangkan pada codingan keempat, algoritma **Insertion Sort** dikembangkan dalam bentuk **GUI (Graphical User Interface)** menggunakan Java Swing. Melalui GUI tersebut, mahasiswa dapat memasukkan data angka, melihat proses pengurutan secara bertahap, serta memahami cara kerja algoritma insertion sort melalui tampilan visual dan log langkah.

Dengan menggabungkan implementasi sorting berbasis console dan GUI, praktikum ini membantu mahasiswa memahami konsep sorting tidak hanya secara teori, tetapi juga melalui penerapan langsung dalam program yang lebih interaktif.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar algoritma sorting menggunakan Bubble Sort, Insertion Sort, dan Selection Sort.
2. Menerapkan algoritma sorting pada data bertipe integer menggunakan array di Java.
3. Mengetahui perbedaan cara kerja masing-masing algoritma sorting melalui hasil program.
4. Memahami alur proses insertion sort secara bertahap melalui visualisasi GUI.
5. Melatih kemampuan pemrograman Java, baik berbasis console maupun GUI menggunakan Java Swing.
6. Membiasakan penggunaan perulangan, percabangan, dan manipulasi array dalam proses sorting.

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini adalah:

1. Mahasiswa dapat memahami cara kerja algoritma sorting secara langsung melalui implementasi program.
2. Mahasiswa dapat membandingkan hasil dan proses pengurutan dari Bubble Sort, Insertion Sort, dan Selection Sort.
3. Mahasiswa menjadi lebih terbiasa menggunakan array untuk mengelola data dalam Java.
4. Mahasiswa mendapatkan pengalaman membuat aplikasi GUI sederhana menggunakan Java Swing.
5. Visualisasi langkah per langkah pada insertion sort membantu mahasiswa memahami logika sorting dengan lebih jelas.
6. Praktikum ini melatih mahasiswa untuk menghubungkan konsep teori struktur data dengan penerapan nyata dalam program.

BAB II PEMBAHASAN

2.1 Langkah Kerja Praktikum

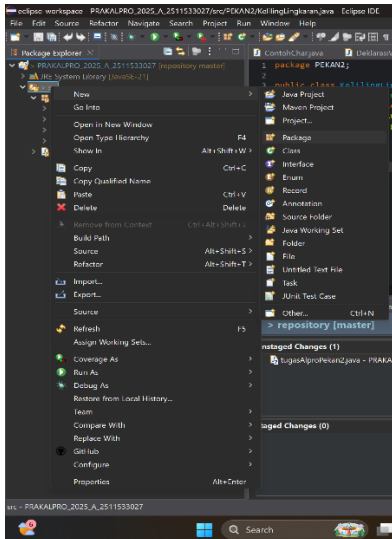
1. Persiapan Awal

- a) Buka apl Eclipse IDE di laptop atau di komputer.

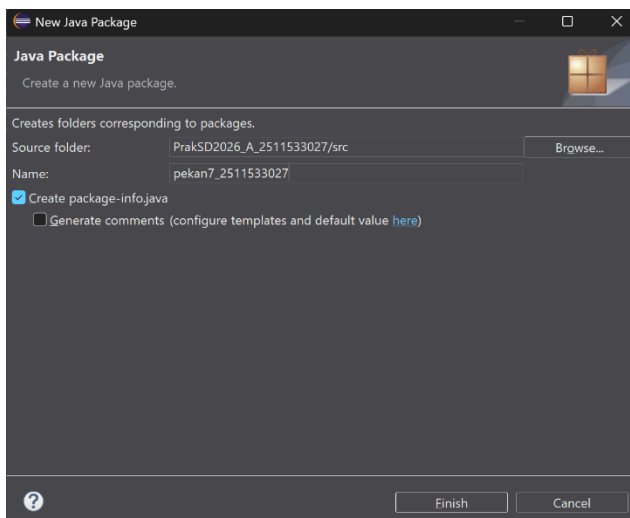


2. Membuat Package

- a) Klik kanan pada folder **src**, lalu pilih **New**, terakhir klik **Package**.

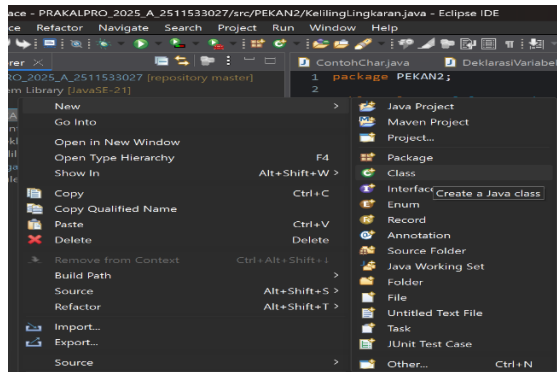


- b) Beri nama package **pekan7_NIM**.



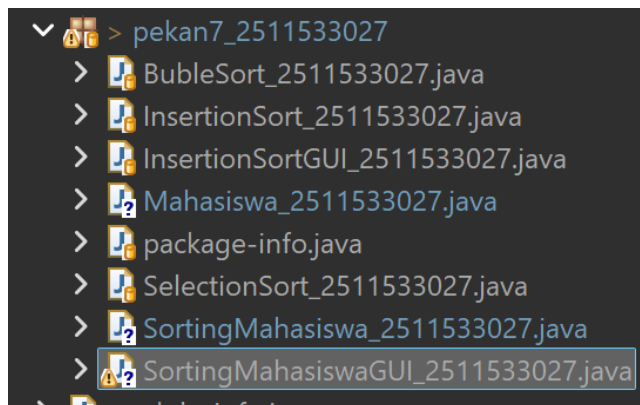
3. Membuat Class Baru untuk Setiap Percobaan

a) Klik kanan pada package **pekan7_NIM**, lalu pilih **New**, terakhir klik **Class**.



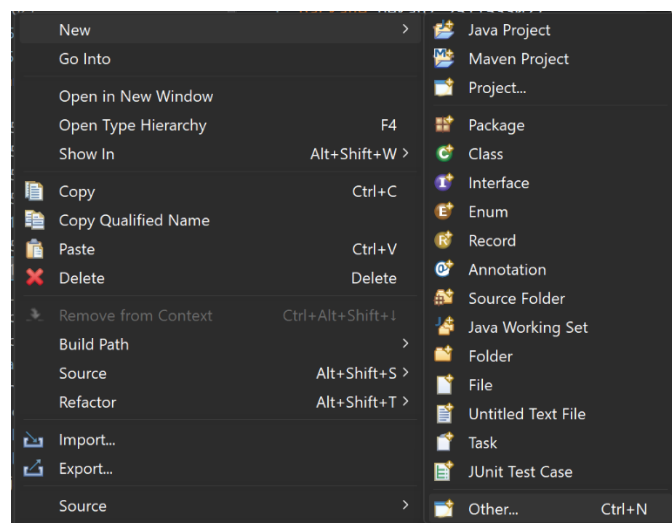
b) Buat beberapa kelas sesuai percobaan.

- BubleSort_2511533027.java
- InsertionSort_2511533027.java
- SelectionSort_2511533027.java

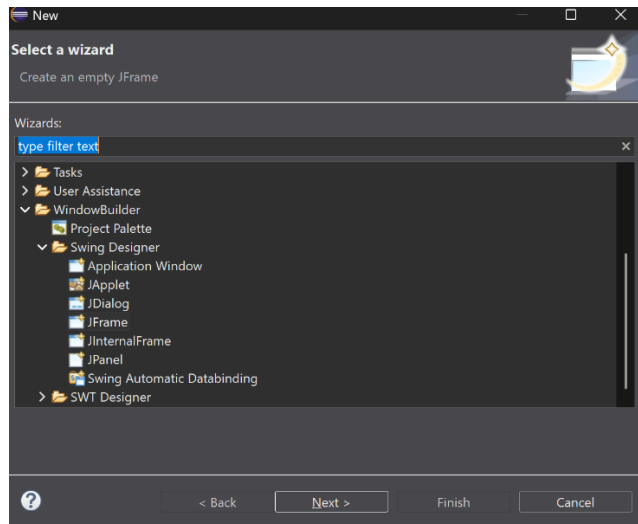


c) Khusus untuk Class GUI (InsertionSortGUI_2511533027.java)

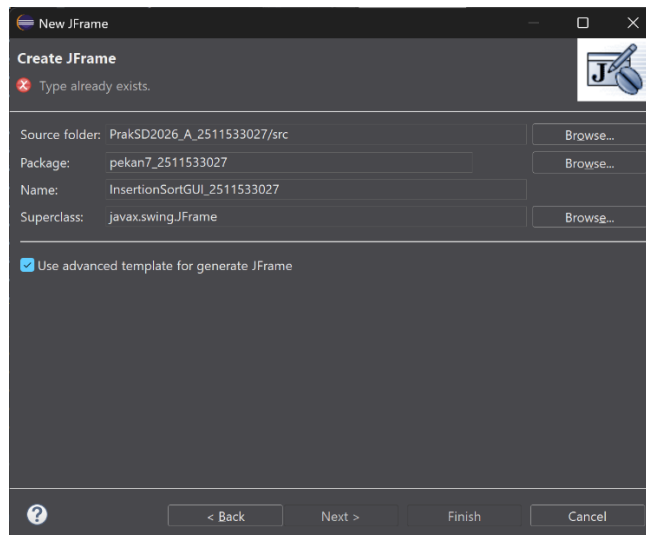
- Klik kanan pada **pekan7_NIM** lalu klik **New** terakhir klik **Other**



- Lalu di **Swing Designer** klik **JFrame**



- Lalu beri nama sesuai instruksi yaitu **InsertionSortGUI_NIM**



4. Menulis Kode Program

1. BubleSort_2511533027.java

- Program ini digunakan untuk memahami proses pengurutan data menggunakan algoritma **Bubble Sort**.
- Program bekerja dengan cara membandingkan dua elemen yang bersebelahan dan menukarnya jika urutannya salah.
- Method bubbleSort_3027() digunakan untuk melakukan proses pengurutan array secara menaik (ascending).
- Proses pengurutan dilakukan menggunakan dua buah perulangan (loop) untuk membandingkan setiap elemen dalam array.
- Pada method main, program mendeklarasikan sebuah array integer yang belum terurut.

- Program menampilkan data array sebelum dilakukan pengurutan dan setelah proses Bubble Sort selesai.
- Hasil akhir program berupa array yang telah terurut menggunakan algoritma Bubble Sort.

```
package pekan7_2511533027;

public class BubleSort_2511533027 {
    public static void bubbleSort_3027(int[] arr_3027) {
        int n_3027 = arr_3027.length;
        for (int i_3027 = 0; i_3027 < n_3027; i_3027++) {
            for (int j_3027 = 0; j_3027 < n_3027 - i_3027 - 1;
j_3027++) {
                if (arr_3027[j_3027] > arr_3027[j_3027 + 1]) {
                    int temp_3027 = arr_3027[j_3027];
                    arr_3027[j_3027] = arr_3027[j_3027 + 1];
                    arr_3027[j_3027 + 1] = temp_3027;
                    // System.out.println("data: "+arr_3027[j_3027]+
"+arr[j_3027+1]);
                }
            }
        }
    }

    public static void main(String[] args) {
        int arr_3027[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_3027 = arr_3027.length;
        System.out.print("array yang belum terurut:");
        for (int i_3027 = 0; i_3027 < n_3027; i_3027++)
            System.out.print(arr_3027[i_3027] + " ");
        System.out.println("");
        bubbleSort_3027(arr_3027);
        System.out.print("array yang terurut menggunakan BubleSort:");
        for (int i_3027 = 0; i_3027 < n_3027; i_3027++)
            System.out.print(arr_3027[i_3027] + " ");
        System.out.println("");
    }
}
```

2. InsertionSort_2511533027.java

- Program ini digunakan untuk memahami proses pengurutan data menggunakan algoritma **Insertion Sort**.
- Algoritma insertion sort bekerja dengan cara mengambil satu elemen dan menempatkannya pada posisi yang sesuai di bagian array yang sudah terurut.
- Method `insertionSort_3027()` digunakan untuk mengatur posisi setiap elemen array dengan cara membandingkannya dengan elemen sebelumnya.
- Proses pengurutan dilakukan menggunakan perulangan `for` dan `while` untuk menggeser elemen yang lebih besar.
- Pada method `main`, program mendeklarasikan array integer sebagai data awal.
- Program menampilkan array sebelum pengurutan dan setelah proses Insertion Sort selesai.
- Hasil akhir program berupa array yang telah tersusun secara terurut.

```
package pekan7_2511533027;

public class InsertionSort_2511533027 {
    public static void insertionSort_3027 (int[] arr_3027) {
        int n_3027 = arr_3027.length;
        for (int i_3027 = 1; i_3027 < n_3027; i_3027++) {
            int key_3027 = arr_3027[i_3027];
            int j_3027 = i_3027 - 1;
            while (j_3027 >= 0 && arr_3027[j_3027] > key_3027) {
                arr_3027[j_3027 + 1] = arr_3027[j_3027];
                j_3027--;
            }
            arr_3027[j_3027 + 1] = key_3027;
        }
    }

    public static void main(String[] args) {
        int arr_3027[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_3027 = arr_3027.length;
        System.out.println("array yang belum terurut:");
        for (int i_3027 = 0; i_3027 < n_3027; i_3027++)
```

```

        System.out.print(arr_3027[i_3027] + " ");
        System.out.println("");
        insertionSort_3027(arr_3027);
        System.out.println("array yang terurut:");
        for (int i_3027 = 0; i_3027 < n_3027; i_3027++)
            System.out.print(arr_3027[i_3027] + " ");
        System.out.println("");
    }
}

```

3. SelectionSort_2511533027.java

- Program ini digunakan untuk memahami cara kerja algoritma **Selection Sort**.
- Selection sort bekerja dengan cara mencari nilai terkecil dari array dan menukarnya dengan elemen pada posisi awal.
- Method selectionSort_3027() digunakan untuk menentukan indeks elemen terkecil pada setiap proses perulangan.
- Proses pengurutan dilakukan dengan membandingkan satu elemen dengan seluruh elemen setelahnya.
- Pada method main, program mendeklarasikan array integer yang belum terurut.
- Program menampilkan kondisi array sebelum dan sesudah dilakukan proses Selection Sort.
- Hasil akhir program berupa array integer yang telah terurut secara menaik.

```

package pekan7_2511533027;

public class SelectionSort_2511533027 {
    public static void selectionSort_3027(int[] arr_3027) {
        int n_3027 = arr_3027.length;
        for (int i_3027 = 0; i_3027 < n_3027; i_3027++) {
            int minIndex_3027 = i_3027;
            for (int j_3027 = i_3027 + 1; j_3027 < n_3027;
j_3027++) {
                if (arr_3027[j_3027] < arr_3027[minIndex_3027]) {
                    minIndex_3027 = j_3027;
                }
            }
        }
    }
}

```

```

    }
    int temp_3027 = arr_3027[i_3027];
    arr_3027[i_3027] = arr_3027[minIndex_3027];
    arr_3027[minIndex_3027] = temp_3027;
}
}
public static void main(String[] args) {
    int arr_3027[] = { 23, 78, 45, 8, 32, 56, 1 };
    int n_3027 = arr_3027.length;
    System.out.printf("array yang belum terurut:\n");
    for (int i_3027 = 0; i_3027 < n_3027; i_3027++)
        System.out.print(arr_3027[i_3027] + " ");
    System.out.println("");
    selectionSort_3027(arr_3027);
    System.out.printf("array yang terurut:\n");
    for (int i_3027 = 0; i_3027 < n_3027; i_3027++)
        System.out.print(arr_3027[i_3027] + " ");
    System.out.println("");
}
}
}

```

4. InsertionSortGUI_2511533027.java

- Program dibuat menggunakan **Java Swing** dengan tampilan utama berupa frame yang berisi input data, visual array, tombol kontrol, dan area log proses.
- **TextField (inputField_3027)** digunakan sebagai tempat user memasukkan data angka yang akan diurutkan. Data dimasukkan dalam satu baris dengan format angka yang dipisahkan oleh tanda koma.
- **JBUTTON Set Array (setButton_3027)** digunakan untuk membaca data dari TextField, mengubahnya menjadi array integer, lalu menampilkan setiap elemen array dalam bentuk label.
- **JPanel (panelArray_3027)** digunakan untuk menampilkan visual array. Setiap elemen array ditampilkan menggunakan **JLabel** sehingga pengguna dapat melihat perubahan posisi data saat proses sorting berlangsung.

- **JLabel (labelArray_3027)** berfungsi untuk menampilkan nilai setiap elemen array secara visual, lengkap dengan border agar terlihat seperti kotak data.
- **JButton Langkah Selanjutnya (stepButton_3027)** digunakan untuk menjalankan proses insertion sort **satu langkah demi satu langkah**, sehingga pengguna dapat memahami alur pengurutan secara bertahap.
- **JTextArea (stepArea_3027)** digunakan untuk menampilkan log atau catatan setiap langkah sorting, seperti data yang sedang diproses dan hasil array setelah setiap langkah selesai.
- **JScrollPane** digunakan agar JTextArea dapat di-scroll ketika jumlah log sudah banyak.
- **JButton Reset (resetButton_3027)** digunakan untuk mengembalikan program ke kondisi awal dengan cara menghapus input, tampilan array, dan isi log proses sorting.
- Proses sorting dikendalikan menggunakan variabel indeks `i_3027` dan `j_3027`, yang mewakili proses insertion sort sesuai konsep teori.
- Setiap kali tombol langkah ditekan, program akan memindahkan satu elemen ke posisi yang sesuai dan memperbarui tampilan label serta isi JTextArea.
- Program akan menampilkan pesan bahwa sorting selesai ketika seluruh data sudah terurut.
- Hasil akhir dari program ini adalah visualisasi proses **Insertion Sort langkah demi langkah**, sehingga pengguna tidak hanya melihat hasil akhir, tetapi juga memahami proses pengurutan secara detail.

```
package pekan7_2511533027;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.BorderFactory;
import javax.swing.JButton;
```

```

import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class InsertionSortGUI_2511533027 extends JFrame {
    private static final long serialVersionUID = 1L;
    private int[] array_3027;
    private JLabel[] labelArray_3027;
    private JButton stepButton_3027, resetButton_3027, setButton_3027;
    private JTextField inputField_3027;
    private JPanel panelArray_3027;
    private JTextArea stepArea_3027;

    private int i_3027 = 1, j_3027;
    private boolean sorting_3027 = false;
    private int stepCount_3027 = 1;

    /**
     * Launch the application.
     */

    /**
     * Create the frame.
     */
    public InsertionSortGUI_2511533027() {
        setTitle("Insertion Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_3027 = new JPanel(new FlowLayout());

```

```

        inputField_3027 = new JTextField(30);
        setButton_3027 = new JButton("Set Array");
        inputPanel_3027.add(new JLabel("Masukkan angka (pisahkan
dengan koma):"));
        inputPanel_3027.add(inputField_3027);
        inputPanel_3027.add(setButton_3027);

// Panel Array visual
panelArray_3027 = new JPanel();
panelArray_3027.setLayout(new FlowLayout());

// Panel kontrol
JPanel controlPanel_3027 = new JPanel();
stepButton_3027 = new JButton("Langkah Selanjutnya");
resetButton_3027 = new JButton("Reset");
stepButton_3027.setEnabled(false);
controlPanel_3027.add(stepButton_3027);
controlPanel_3027.add(resetButton_3027);

// Area text untuk log langkah-langkah
stepArea_3027 = new JTextArea(8, 60);
stepArea_3027.setEditable(false);
stepArea_3027.setFont(new Font("Monospaced", Font.PLAIN, 14));
JScrollPane scrollPane_3027 = new JScrollPane(stepArea_3027);

// Tambahkan panel ke frame
add(inputPanel_3027, BorderLayout.NORTH);
add(panelArray_3027, BorderLayout.CENTER);
add(controlPanel_3027, BorderLayout.SOUTH);
add(scrollPane_3027, BorderLayout.EAST);

// Event Set Array
setButton_3027.addActionListener(e ->
setArrayFromInput_3027());

// Event Langkah Selanjutnya
stepButton_3027.addActionListener(e -> performStep_3027());

// Event Reset
resetButton_3027.addActionListener(e -> reset_3027());

```

```

    }

    private void setArrayFromInput_3027() {
        String text_3027 = inputField_3027.getText().trim();
        if (text_3027.isEmpty()) return;
        String[] parts_3027 = text_3027.split(",");
        array_3027 = new int[parts_3027.length];
        try {
            for (int k_3027 = 0; k_3027 < parts_3027.length;
k_3027++) {
                array_3027[k_3027] =
Integer.parseInt(parts_3027[k_3027].trim()); }
            } catch (NumberFormatException e) {
                JOptionPane.showMessageDialog(this, "Masukkan hanya
angka yang dipisahkan " + "dengan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
                return; }
        i_3027 = 1;
        stepCount_3027 = 1;
        sorting_3027 = true;
        stepButton_3027.setEnabled(true);
        stepArea_3027.setText("");
        panelArray_3027.removeAll();
        labelArray_3027 = new JLabel[array_3027.length];
        for (int k_3027 = 0; k_3027 < array_3027.length; k_3027++)
{
            labelArray_3027[k_3027] = new
JLabel(String.valueOf(array_3027[k_3027]));
            labelArray_3027[k_3027].setFont(new Font("Arial",
Font.BOLD, 24));

labelArray_3027[k_3027].setBorder(BorderFactory.createLineBorder(Color.BLAC
K));

            labelArray_3027[k_3027].setPreferredSize(new
Dimension(50, 50));

labelArray_3027[k_3027].setHorizontalAlignment(SwingConstants.CENTER);
            panelArray_3027.add(labelArray_3027[k_3027]);
        }

        panelArray_3027.revalidate();
    }

```



```

        panelArray_3027.repaint();
    }

    private void performStep_3027() {
        if (i_3027 < array_3027.length && sorting_3027) {
            int key_3027 = array_3027[i_3027];
            j_3027 = i_3027 - 1;

            StringBuilder stepLog_3027 = new StringBuilder();
            stepLog_3027.append("Langkah
").append(stepCount_3027).
            append(": Memasukkan ").append(key_3027).append("\n");

            while (j_3027 >= 0 && array_3027[j_3027] > key_3027) {
                array_3027[j_3027 + 1] = array_3027[j_3027];
                j_3027--;
            }
            array_3027[j_3027 + 1] = key_3027;

            updateLabels_3027();
            stepLog_3027.append("Hasil:
").append(arrayToString_3027(array_3027)).append("\n\n");
            stepArea_3027.append(stepLog_3027.toString());

            i_3027++;
            stepCount_3027++;

            if (i_3027 == array_3027.length) {
                sorting_3027 = false;
                stepButton_3027.setEnabled(false);
                JOptionPane.showMessageDialog(this, "Sorting
selesai!");
            }
        }
    }

    private void updateLabels_3027() {
        for (int k_3027 = 0; k_3027 < array_3027.length; k_3027++)
{

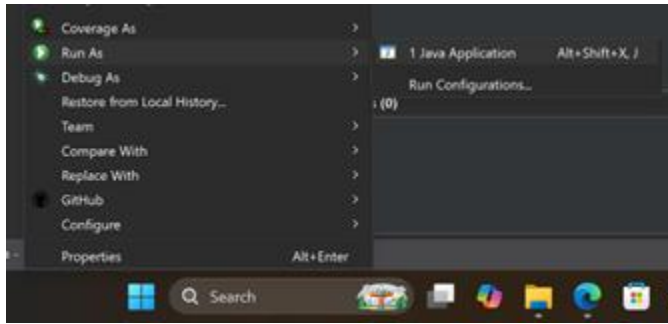
```

```

labelArray_3027[k_3027].setText(String.valueOf(array_3027[k_3027]));
    }
}
private void reset_3027() {
    inputField_3027.setText("");
    panelArray_3027.removeAll();
    panelArray_3027.revalidate();
    panelArray_3027.repaint();
    stepArea_3027.setText("");
    stepButton_3027.setEnabled(false);
    sorting_3027 = false;
    i_3027 = 1;
    stepCount_3027 = 1;
}
private String arrayToString_3027(int[] arr_3027) {
    StringBuilder sb_3027 = new StringBuilder();
    for (int k_3027 = 0; k_3027 < arr_3027.length; k_3027++) {
        sb_3027.append(arr_3027[k_3027]);
        if (k_3027 < arr_3027.length - 1) sb_3027.append(",
");
    }
    return sb_3027.toString();
}
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        InsertionSortGUI_2511533027 gui_3027 = new
InsertionSortGUI_2511533027();
        gui_3027.setVisible(true);
    });
}
}

```

5. Kompilasi dan Menjalankan Program



2.2 Analisis Hasil dan Pembahasan

1. BubleSort_2511533027.java

- a) Hasil: Program berhasil melakukan proses pengurutan data menggunakan algoritma Bubble Sort. Data array yang awalnya belum terurut dapat diurutkan secara menaik setelah method bubbleSort_3027() dijalankan. Program juga berhasil menampilkan isi array sebelum dan sesudah proses pengurutan melalui output di console.
- b) Pembahasan: Pada program ini, algoritma Bubble Sort bekerja dengan cara membandingkan dua elemen yang bersebelahan di dalam array. Jika elemen pada indeks sebelumnya lebih besar dari elemen setelahnya, maka kedua elemen tersebut ditukar. Proses ini dilakukan secara berulang menggunakan dua perulangan hingga seluruh data berada pada posisi yang benar. Dari hasil percobaan dapat dilihat bahwa setiap iterasi akan mendorong nilai terbesar ke bagian akhir array, sehingga pada akhirnya array menjadi terurut secara menaik.

2. InsertionSort_2511533027.java

- a) Hasil: Program berhasil mengurutkan data array integer menggunakan algoritma Insertion Sort. Array yang awalnya belum terurut dapat disusun dengan benar setelah method insertionSort_3027() dipanggil. Hasil pengurutan dapat dilihat melalui tampilan array sebelum dan sesudah sorting pada console.
- b) Pembahasan: Pada program ini, Insertion Sort bekerja dengan mengambil satu elemen sebagai key, kemudian membandingkannya dengan elemen-elemen sebelumnya. Jika ditemukan elemen yang lebih besar dari key, maka elemen tersebut digeser ke kanan hingga posisi yang sesuai ditemukan. Proses ini dilakukan secara bertahap mulai dari indeks kedua array. Berdasarkan hasil yang diperoleh, insertion sort mampu mengurutkan data secara efektif dengan cara menyisipkan setiap elemen ke posisi yang tepat pada bagian array yang sudah terurut.

3. SelectionSort_2511533027.java

- a) Hasil: Program berhasil melakukan pengurutan data menggunakan algoritma Selection Sort. Data array yang belum terurut dapat disusun menjadi terurut secara menaik setelah proses sorting dijalankan. Output program menampilkan kondisi array sebelum dan sesudah pengurutan dengan benar.
- b) Pembahasan: Pada program ini, Selection Sort bekerja dengan mencari elemen terkecil dari bagian array yang belum terurut. Setelah elemen terkecil ditemukan, nilai tersebut ditukar dengan elemen pada posisi awal bagian array tersebut. Proses ini dilakukan secara berulang hingga seluruh array terurut. Dari hasil percobaan dapat dilihat bahwa setiap iterasi akan menempatkan satu elemen terkecil ke posisi yang benar, sehingga jumlah data yang belum terurut semakin berkurang.

4. InsertionSortGUI_2511533027.java

- a) Hasil: Program berhasil menampilkan proses algoritma Insertion Sort dalam bentuk GUI. Pengguna dapat memasukkan data angka melalui input, kemudian melihat proses pengurutan secara bertahap menggunakan tombol Langkah Selanjutnya. Setiap langkah sorting ditampilkan secara visual melalui label array dan dicatat pada area log. Program juga berhasil mereset data dan tampilan menggunakan tombol Reset.
- b) Pembahasan: Pada program GUI ini, algoritma Insertion Sort diterapkan dengan cara yang sama seperti versi console, namun ditampilkan secara visual. Data yang dimasukkan melalui JTextField diproses menjadi array integer dan divisualisasikan menggunakan JLabel di dalam JPanel. Setiap kali tombol langkah ditekan, satu proses penyisipan data dilakukan dan hasilnya diperbarui pada tampilan array serta dicatat pada JTextArea sebagai log langkah. Dengan adanya visualisasi dan log proses, pengguna dapat memahami bagaimana setiap elemen dipindahkan hingga array menjadi terurut. Tombol reset berfungsi untuk mengembalikan program ke kondisi awal sehingga proses sorting dapat diulang dari awal.

BAB III

PENUTUP

3.1 Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting memiliki peran penting dalam mengatur data agar menjadi lebih rapi dan mudah dipahami. Melalui praktikum ini, mahasiswa dapat melihat secara langsung bagaimana proses pengurutan data bekerja menggunakan tiga algoritma berbeda, yaitu Bubble Sort, Insertion Sort, dan Selection Sort. Setiap algoritma memiliki cara kerja yang berbeda, tetapi tujuannya sama, yaitu mengurutkan data dari nilai terkecil ke terbesar.

Pada implementasi program berbasis console, mahasiswa dapat memahami perbedaan hasil pengurutan sebelum dan sesudah sorting dilakukan. Sedangkan melalui program berbasis GUI, proses Insertion Sort menjadi lebih mudah dipahami karena setiap langkah pengurutan ditampilkan secara visual dan disertai dengan penjelasan pada log proses. Hal ini sangat membantu dalam memahami alur perpindahan data selama proses sorting berlangsung.

Secara keseluruhan, praktikum ini membantu mahasiswa untuk tidak hanya memahami teori sorting, tetapi juga mampu menerapkannya langsung ke dalam program Java. Dengan adanya praktikum ini, mahasiswa menjadi lebih paham cara kerja algoritma sorting dan lebih siap untuk mempelajari materi struktur data selanjutnya.

DAFTAR PUSTAKA

- [1] Wahyudi, *Materi Struktur Data: Sorting*. Padang: PPT Perkuliahan, 2026.
- [2] Oracle Corporation, *Java Platform, Standard Edition Documentation*. [Online]. Available: <https://docs.oracle.com/javase/8/docs/>

LAPORAN TUGAS PRAKTIKUM

STRUKTUR DATA

“SORTING”



disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Dengan memanjatkan puji syukur kehadiran Allah SWT, atas rahmat dan karunia-Nya sehingga laporan tugas praktikum Struktur Data ini dapat diselesaikan dengan baik. Shalawat serta salam semoga selalu tercurah kepada Nabi Muhammad SAW.

Laporan tugas ini disusun untuk memenuhi tugas praktikum Struktur Data dengan topik Sorting menggunakan studi kasus Pengurutan Nama Mahasiswa Secara Alfabetis Menggunakan GUI. Pada praktikum ini, penulis membuat program Java yang dapat mengelola data nama mahasiswa dan dapat mengurutkannya dengan Algoritma Sorting (Insertion Sort, Selection Sort, dan Bubble Sort).

Penulis mengucapkan terima kasih kepada dosen pengampu dan asisten praktikum atas bimbingannya. Penulis juga menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran sangat diharapkan.

DAFTAR ISI

DAFTAR ISI.....	ii
BAB I.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	2
1.4 Instruksi Tugas	2
BAB II	4
2.1 Kelas ADT Mahasiswa	4
2.2 Kelas Sorting Mahasiswa.....	5
2.3 Kelas GUI Utama.....	6
BAB III.....	9
3.1 Kesimpulan	9
DAFTAR PUSTAKA	

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia perkuliahan, khususnya di bidang informatika, pengolahan data merupakan hal yang sangat sering dilakukan. Salah satu contoh data yang sering diolah adalah data mahasiswa, seperti nama, NIM, dan program studi. Data-data tersebut akan lebih mudah dibaca dan dipahami jika disusun secara teratur, misalnya berdasarkan nama mahasiswa.

Melalui praktikum ini, mahasiswa diperkenalkan dengan konsep sorting menggunakan bahasa pemrograman Java. Tidak hanya sekedar mengurutkan angka, praktikum ini juga menerapkan algoritma sorting pada objek, yaitu data mahasiswa. Selain itu, proses sorting juga ditampilkan dalam bentuk antarmuka grafis (GUI), sehingga mahasiswa dapat melihat langsung proses pengurutan data yang terjadi.

Dengan adanya program berbasis GUI ini, mahasiswa tidak hanya memahami hasil akhir dari sorting, tetapi juga memahami proses di balik pengurutan data. Hal ini diharapkan dapat membantu mahasiswa dalam memahami konsep algoritma sorting secara lebih jelas dan aplikatif.

1.2 Tujuan

Tujuan dari pembuatan tugas ini adalah:

1. Untuk memahami konsep dasar algoritma sorting menggunakan Bubble Sort, Selection Sort, dan Insertion Sort.
2. Untuk mengimplementasikan algoritma sorting pada data bertipe objek, yaitu data mahasiswa.
3. Untuk memahami penggunaan ArrayList dalam menyimpan dan mengelola data.
4. Untuk memahami penggunaan method `compareToIgnoreCase()` dalam membandingkan data bertipe String.
5. Untuk mengenal dan menerapkan konsep GUI menggunakan Java Swing dalam sebuah program.
6. Untuk melihat proses sorting data secara bertahap melalui tampilan log pada GUI.

1.3 Manfaat

Manfaat dari pembuatan tugas ini adalah:

1. Mahasiswa menjadi lebih paham cara kerja algoritma sorting dalam pemrograman Java.
2. Mahasiswa dapat mengurutkan data mahasiswa berdasarkan nama secara otomatis.
3. Mahasiswa dapat memahami perbedaan cara kerja Bubble Sort, Selection Sort, dan Insertion Sort.
4. Mahasiswa mendapatkan pengalaman dalam membuat program berbasis GUI menggunakan Java Swing.
5. Mahasiswa dapat melihat dan memahami proses sorting secara langsung melalui tampilan log, bukan hanya melihat hasil akhirnya saja.
6. Praktikum ini membantu mahasiswa untuk lebih siap dalam mempelajari materi struktur data dan algoritma pada tahap selanjutnya.

1.4 Instruksi Tugas

1. Membuat Kelas ADT Mahasiswa

Class digunakan untuk menyimpan data mahasiswa.

Atribut:

- Nama Mahasiswa (String)
- NIM Mahasiswa (String)
- Program Studi (String)

Method:

- Constructor
- Getter dan Setter
- toString()

2. Kelas GUI Utama

Class GUI digunakan untuk:

- Menampilkan form input data.
- Menampilkan tabel data mahasiswa.
- Menampilkan ComboBox pilihan algoritma sorting.
- Menampilkan proses visualisasi sorting.
- Menjalankan algoritma sorting sesuai pilihan pengguna.

3. Method Sorting

Program menyediakan tiga method sorting terpisah:

- insertionSort()
- selectionSort()
- bubbleSort()

4. Komponen GUI

GUI program terdiri dari beberapa komponen berikut:

- JTextField Digunakan untuk input nama mahasiswa, NIM, dan program studi.
- JButton Digunakan untuk:
 - Menambahkan data
 - Menghapus data
 - Memulai sorting
- JComboBox Digunakan untuk memilih algoritma sorting:
 - Insertion Sort
 - Selection Sort
 - Bubble Sort
- JTable / JTextArea Digunakan untuk menampilkan:
 - Data mahasiswa
 - Langkah-langkah proses sorting
 - Hasil akhir sortin

5. Contoh Visualisasi Sorting

Setiap algoritma sorting wajib menampilkan proses pengurutan secara bertahap agar alur algoritma dapat dipahami dengan jelas.

Contoh visualisasi (Java GUI (Swing))

=== INSERTION SORT ===

Langkah 1 : [Budi, Rizky, Maya]

Langkah 2 : [Budi, Maya, Rizky]

=== SELECTION SORT ===

Pass 1 : [Andi, Budi, Maya, Rizky]

=== BUBBLE SORT ===

Pass 1 : [Budi, Maya, Andi, Rizky]

BAB II

PEMBAHASAN

2.1 Kelas ADT Mahasiswa

Program ini merupakan class yang digunakan untuk merepresentasikan data mahasiswa. Di dalam class ini terdapat tiga atribut utama, yaitu nama, NIM, dan program studi, yang masing-masing disimpan dalam bentuk tipe data String. Class ini juga dilengkapi dengan constructor untuk menginisialisasi data mahasiswa saat objek dibuat, serta getter dan setter untuk mengakses dan mengubah nilai dari setiap atribut.

Selain itu, class ini memiliki method toString() yang digunakan untuk menampilkan nama mahasiswa ketika objek dicetak atau ditampilkan dalam bentuk teks. Hal ini memudahkan proses sorting dan penampilan data pada program GUI, karena yang ditampilkan cukup nama mahasiswa saja tanpa perlu menuliskan atribut lainnya.

```

package pekan7_2511533027;

public class Mahasiswa_2511533027 {
    private String nama_3027;
    private String nim_3027;
    private String prodi_3027;

    // Constructor
    public Mahasiswa_2511533027(String nama_3027, String nim_3027, String prodi_3027) {
        this.nama_3027 = nama_3027;
        this.nim_3027 = nim_3027;
        this.prodi_3027 = prodi_3027;
    }

    // Getter
    public String getNama_3027() {
        return nama_3027;
    }
    public String getNim_3027() {
        return nim_3027;
    }
    public String getProdi_3027() {
        return prodi_3027;
    }

    // Setter
    public void setNama_3027(String nama_3027) {
        this.nama_3027 = nama_3027;
    }
    public void setNim_3027(String nim_3027) {
        this.nim_3027 = nim_3027;
    }
    public void setProdi_3027(String prodi_3027) {
        this.prodi_3027 = prodi_3027;
    }

    // toString
    @Override
    public String toString() {
        return nama_3027;
    }
}

```

2.2 Kelas Sorting Mahasiswa

Program ini digunakan untuk mengatur proses pengurutan data mahasiswa berdasarkan nama. Data mahasiswa disimpan dalam ArrayList, sehingga memudahkan dalam pengelolaan data yang bersifat dinamis. Program ini mengimplementasikan tiga algoritma sorting, yaitu Bubble Sort, Selection Sort, dan Insertion Sort, yang semuanya membandingkan nama mahasiswa menggunakan method `compareToIgnoreCase()` agar tidak terpengaruh perbedaan huruf besar dan kecil.

Setiap algoritma sorting ditampilkan prosesnya melalui JTextArea dalam bentuk log langkah atau pass. Dengan adanya log ini, pengguna dapat melihat perubahan urutan data

mahasiswa pada setiap langkah sorting. Hal ini membantu dalam memahami cara kerja masing-masing algoritma sorting secara lebih jelas.

```
package pekan7_2511533027;

import java.util.ArrayList;

public class SortingMahasiswa_2511533027 {
    public static void bubbleSort_3027(ArrayList<Mahasiswa_2511533027> data_3027, JTextArea log_3027) {
        int n_3027 = data_3027.size();
        for (int i_3027 = 0; i_3027 < n_3027; i_3027++) {
            for (int j_3027 = 0; j_3027 < n_3027 - i_3027 - 1; j_3027++) {
                if (data_3027.get(j_3027).getNama_3027().compareToIgnoreCase(data_3027.get(j_3027 + 1).getNama_3027()) > 0) {
                    Mahasiswa_2511533027 temp_3027 = data_3027.get(j_3027);
                    data_3027.set(j_3027, data_3027.get(j_3027 + 1));
                    data_3027.set(j_3027 + 1, temp_3027);
                }
            }
            log_3027.append("Pass " + (i_3027 + 1) + " : " + data_3027 + "\n");
        }
    }

    public static void selectionSort_3027(ArrayList<Mahasiswa_2511533027> data_3027, JTextArea log_3027) {
        for (int i_3027 = 0; i_3027 < data_3027.size() - 1; i_3027++) {
            int min_3027 = i_3027;
            for (int j_3027 = i_3027 + 1; j_3027 < data_3027.size(); j_3027++) {
                if (data_3027.get(j_3027).getNama_3027().compareToIgnoreCase(data_3027.get(min_3027).getNama_3027()) < 0) {
                    min_3027 = j_3027;
                }
            }
            Mahasiswa_2511533027 temp_3027 = data_3027.get(min_3027);
            data_3027.set(min_3027, data_3027.get(i_3027));
            data_3027.set(i_3027, temp_3027);
            log_3027.append("Pass " + (i_3027 + 1) + " : " + data_3027 + "\n");
        }
    }

    public static void insertionSort_3027(ArrayList<Mahasiswa_2511533027> data_3027, JTextArea log_3027) {
        for (int i_3027 = 1; i_3027 < data_3027.size(); i_3027++) {
            Mahasiswa_2511533027 key_3027 = data_3027.get(i_3027);
            int j_3027 = i_3027 - 1;
            while (j_3027 >= 0 && data_3027.get(j_3027).getNama_3027().compareToIgnoreCase(key_3027.getNama_3027()) > 0) {
                data_3027.set(j_3027 + 1, data_3027.get(j_3027));
                j_3027--;
            }
            data_3027.set(j_3027 + 1, key_3027);
            log_3027.append("Langkah " + i_3027 + " : " + data_3027 + "\n");
        }
    }
}
```

2.3 Kelas GUI Utama

Program ini merupakan tampilan antarmuka grafis (GUI) yang digunakan untuk mengelola dan mengurutkan data mahasiswa. Pengguna dapat memasukkan data mahasiswa berupa nama, NIM, dan program studi melalui text field yang disediakan, kemudian menambahkan data tersebut ke dalam sistem menggunakan tombol “Tambah Data”. Data yang dimasukkan akan disimpan ke dalam ArrayList dan ditampilkan melalui log.

Program ini juga menyediakan pilihan metode sorting melalui JComboBox, sehingga pengguna dapat memilih algoritma sorting yang diinginkan. Setelah proses sorting dijalankan, hasil dan langkah-langkah sorting akan ditampilkan pada JTextArea. Selain

itu, terdapat tombol reset yang berfungsi untuk menghapus seluruh data dan mengembalikan program ke kondisi awal. Dengan adanya GUI ini, proses sorting data mahasiswa menjadi lebih interaktif dan mudah dipahami.

```
package pekan7_2511533027;

import java.awt.BorderLayout;
import java.awt.GridLayout;
import java.util.ArrayList;

import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingUtilities;

public class SortingMahasiswaGUI_2511533027 extends JFrame {
    /**
     *
     */
    private static final long serialVersionUID = 1L;
    private JTextField nama_3027, nim_3027, prodi_3027;
    private JTextArea log_3027;
    private JButton reset_3027;
    private JComboBox<String> comboSort_3027;
    private ArrayList<Mahasiswa_2511533027> data_3027 = new ArrayList<>();

    /**
     * Launch the application.
     */
    /**
     * Create the frame.
     */
    public SortingMahasiswaGUI_2511533027() {
        setTitle("Sorting Nama Mahasiswa");
        setSize(750, 400);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JPanel inputPanel_3027 = new JPanel(new GridLayout(5, 2));
        nama_3027 = new JTextField();
        nim_3027 = new JTextField();
```



```

prodi_3027 = new JTextField();

inputPanel_3027.add(new JLabel("Nama"));
inputPanel_3027.add(nama_3027);
inputPanel_3027.add(new JLabel("NIM"));
inputPanel_3027.add(nim_3027);
inputPanel_3027.add(new JLabel("Prodi"));
inputPanel_3027.add(prodi_3027);

JButton tambah_3027 = new JButton("Tambah Data");
JButton sort_3027 = new JButton("Mulai Sorting");
reset_3027 = new JButton("Reset");

inputPanel_3027.add(tambah_3027);
inputPanel_3027.add(sort_3027);
inputPanel_3027.add(reset_3027);

comboSort_3027 = new JComboBox<>(new String[] {"Insertion Sort", "Selection Sort", "Bubble Sort"});

log_3027 = new JTextArea();
log_3027.setEditable(false);

add(inputPanel_3027, BorderLayout.NORTH);
add(comboSort_3027, BorderLayout.WEST);
add(new JScrollPane(log_3027), BorderLayout.CENTER);

tambah_3027.addActionListener(e -> {
    data_3027.add(new Mahasiswa_2511533027(
        nama_3027.getText(),
        nim_3027.getText(),
        prodi_3027.getText()
    ));
    log_3027.append("Data Ditambahkan: " + nama_3027.getText() + "\n");
    nama_3027.setText("");
    nim_3027.setText("");
    prodi_3027.setText("");
});

sort_3027.addActionListener(e -> {
    log_3027.append("\n== PROSES SORTING ==\n");

```

```

String pilihan_3027 = comboSort_3027.getSelectedItem().toString();

if (pilihan_3027.equals("Insertion Sort")) {
    SortingMahasiswa_2511533027.insertionSort_3027(data_3027, log_3027);
} else if (pilihan_3027.equals("Selection Sort")) {
    SortingMahasiswa_2511533027.selectionSort_3027(data_3027, log_3027);
} else {
    SortingMahasiswa_2511533027.bubbleSort_3027(data_3027, log_3027);
}

log_3027.append("\nHasil Akhir: " + data_3027 + "\n");
});

reset_3027.addActionListener(e -> {
    nama_3027.setText("");
    nim_3027.setText("");
    prodi_3027.setText("");
    data_3027.clear();
    log_3027.setText("");
    log_3027.append("Semua Data Berhasil di Reset.\n");
});
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new SortingMahasiswaGUI_2511533027().setVisible(true);
    });
}
}

```

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum dan pembuatan program yang telah dilakukan, dapat disimpulkan bahwa konsep **struktur data dan algoritma sorting** sangat penting dalam pengolahan data agar tersusun secara teratur dan mudah diakses. Melalui implementasi kode program, mahasiswa dapat memahami bagaimana data diproses langkah demi langkah, mulai dari pembandingan hingga pertukaran elemen, sesuai dengan algoritma sorting yang digunakan.

Selain itu, penggunaan bahasa pemrograman **Java** membantu mahasiswa memahami penerapan konsep teori ke dalam bentuk program nyata. Dengan melakukan pengujian program, dapat dilihat bahwa algoritma sorting mampu meningkatkan efisiensi pencarian dan pengelolaan data. Praktikum ini tidak hanya melatih logika berpikir, tetapi juga meningkatkan pemahaman terhadap struktur data sebagai dasar penting dalam pengembangan perangkat lunak.

DAFTAR PUSTAKA

- [1] Wahyudi, *Materi Struktur Data: Sorting*. Padang: PPT Perkuliahan, 2026.
- [2] Oracle Corporation, *Java Platform, Standard Edition Documentation*. [Online]. Available: <https://docs.oracle.com/javase/8/docs/>